calling for you. The most common case is `id=object_id`, searching for a single object that has the corresponding ID, usually passed in as an argument to the view.

Why 404?

If the arguments to `get_object_or_404()` don't result in a single object, either the arguments match multiple arguments or one isn't found, the shortcut raises an `Http404` exception. Django automatically handles this, showing a `404` page instead of the result of the view.

This mechanism is rather important. Although your site probably won't show links with bad arguments, URLs are part of the public space: your users could type any argument at all. You should never trust your users, and your view should always be able to handle bad arguments. Raising an `Http404` error is a convenient way of short-circuiting your view and letting Django do the work.

Django provides a default, somewhat plain `404` page, but you can customize your own for your site. See appendix B on using Django in production environments to learn how.

Once an object gets found, the rest of the view is similar to the views you've written before. You use the `render()` shortcut to render a template, passing in your object as context. The following listing is a view that looks up a `Musician` object by ID and renders RiffMates/templates/musician.html.

Listing 4.4 A view for displaying a `Musician` object based on an argument

```
# RiffMates/bands/views.py
from django.shortcuts import render, get_object_or_404

from bands.models import Musician

def musician(request, musician_id):
    musician = get_object_or_404(Musician, id=musician_id)

    data = {
        "musician": musician,
    }

    return render(request, "musician.html", data)
```

Import get_object_or_404().

Import the Musician object to be queried.

Find a Musician object with the argument's ID or raise a 404 error.

Add the Musician object to the context dictionary.

Like your other views, this one needs to be registered. The `path` object is a little different from those you've seen before. This one includes a capture specifier to indicate that part of the URL is to be used as an argument. Capture specifiers are inside of angle brackets and consist of the type of the argument and the name used in the view. For example, `<int:musician_id>` indicates the argument is an integer and is named `musician_id` in the view. All URLs are text; by including the data type in the capture specifier, Django can automatically cast the given argument to your desired data type.